

CSE 144 Final Project Report

Transfer Learning Challenge

Group Members (max 3):

Alexander Santiago (CruzID: 1995230)

Spring 2026

1 Introduction

1. Problem goal and setting.

This is a 100-class image classification task using transfer learning. The goal is to take a model pretrained on ImageNet and fine-tune it so it can classify images into classes 0 - 99. The dataset is small with about 1,079 labeled training images (~ 10 per class) and ~ 1,000 unlabeled test images for Kaggle submission. With this little data, training a large model from scratch would be slow, data-hungry, and likely to overfit. Transfer learning is a better fit where the pretrained model already knows general features, and we only need to adapt the final layers to our 100 classes. Random guessing would give ~ 1% accuracy, so any meaningful result requires the model to actually learn class-specific patterns.

2. Why transfer learning is appropriate.

Instead of learning all features from ~ 1,000 images, we start from weights that already work well on natural images. Pre-trained models have this advantage and is the approach we are taking. We freeze most of the network and train only the classifier head first, then optionally unfreeze a small part of the backbone with a lower learning rate. Since many models can be really large we don't want to retrain everything but instead use a smaller learning rate so that it can adapt to our use case. This reduces training time and improves generalization compared to training from scratch on a tiny dataset.

3. Brief summary of your approach and main result.

I started with ResNet18 (head-only, then head + last block) and reached roughly ~ 43 - 56% validation accuracy and ~ 0.54 - 0.57% Kaggle score after fixing a label-mapping bug. I

also tried EfficientNet-B3, where unfreezing the last block often caused more overfitting without beating head-only training.

My final model is ViT-B/16, trained with a two-phase schedule (head only → last encoder block + head), strong data augmentation, MixUp, label smoothing, and dropout. For hyperparameter tuning I used an 80/20 train/validation split until I was satisfied, for the final Kaggle submission I retrained on all 1,079 training images to get better results. Inference uses 10-crop test-time augmentation (TTA) to write the submission.csv.

2 Dataset

1. Number of classes and dataset sizes (train/val/test).

Split	Size	Purpose
Train (full)	1,079 images	All labeled data in data/train/
Train (80% split)	~ 863 images	Used during development
Validation (20% split)	~ 216 images	Local tuning for hyperparameters
Test (Kaggle)	~ 1,000 images	Unlabeled dataset in data/test/ , used for Kaggle submission

There are 100 classes (folders 0 through 99), with roughly 10 images per class in training. The 80/20 split was useful for comparing learning rates, dropout, and fine-tuning strategies without submitting to Kaggle every time. Validation accuracy was somewhat noisy (due to small val set) but gave input for whether changes helped. Before the final submission, I switched back to training on the full training set to use all available labels and get a better score.

2. Directory structure and label mapping details (ensure labels 0–99 match folders).

The directory structure was as follows

data/

test/

0.jpg

1.jpg

```
...
train/
  0/
    0.jpg
    ...
    9.jpg
  1/
    10.jpg
    ...
  19.jpg
```

Each folder name is the integer class label (folder 0 → label 0, ..., folder 99 → label 99). I ran into a bug where PyTorch's default ImageFolder sorts class folders lexicographically ("0", "1", "10", "11", ..., "2"), not numerically. That misaligned model outputs with Kaggle labels and produced very low scores (~17%) even when the model was learning. This gave the impression of random guessing.

I fixed this with a custom NumericImageFolder that sorts folders by `int(name)` and sets `class_to_idx[name] = int(name)`. At inference, predictions are mapped back to Kaggle labels using the saved `class_to_idx` from the checkpoint. Verifying this mapping was one of the most important early steps to ensure consistent submissions.

3. Preprocessing (resize, normalization) and data augmentation used

For preprocessing, I followed the standard ImageNet setup that pretrained models expect. Images are resized to 256 pixels, then cropped to 224×224. For training I used random crops, and for validation, test, and Kaggle inference I used a center crop so results are consistent. After that, I normalized the pixel values using ImageNet mean and standard deviation found on the PyTorch docs for each respective model. This matters because the pretrained weights were trained with the same normalization, so using different values would hurt the performance.

During training, I also used data augmentation to help the model generalize with only about 1,000 images. The main augmentations were random resized crops, horizontal flips, small rotations, color jitter, occasional grayscale conversion, and random erasing. The idea was to show the model slightly different versions of the same image so it learns useful features instead of memorizing exact training photos, which in turn reduces overfitting and improves

performance. For validation, I turned augmentation off and only used resize, center crop, and normalization. That way validation accuracy reflects how the model performs on clean images, not randomly transformed ones.

At inference time for Kaggle, I used 10-crop test-time augmentation. The image is resized, split into five crops, and each crop is also flipped horizontally. The model makes a prediction on all 10 versions, and I combine those predictions before choosing the final label. This made inference slower, but it usually gave a small improvement over using a single center crop.

3 Implementation

3.1 Model

1. Pretrained backbone used (e.g., ResNet/EfficientNet/ViT) and why.

Model	Role
ResNet18	Initial model used for baseline
ResNet50	Larger model tested to see improvement in accuracy
EfficientNet-B3	Stronger backbone to use. Head only training worked more effectively.
ViT-B/16 (Best Model)	Vision Transformer pretrained on ImageNet.

I started with ResNet18 because it is lightweight and well suited to small-data transfer learning. Head-only training with tuned learning rate and weight decay reached ~ 0.43 – 0.51 validation accuracy. Unfreezing layer4 (last residual block) improved validation to ~ 56% but increased overfitting (train accuracy near 94%).

2. Architecture changes (classifier head, pooling, dropout, etc.).

For all models, I replaced the original classifier with:

Dropout(p=0.3) → Linear(in_features → 100)

I tested dropout values from 0.1 to 0.5. 0.3 worked best as a balance between regularization and capacity. The behavior I saw when increasing it past 0.3 was that it reduced overfitting but also prevented improvement in validation accuracy.

For ViT-B/16, the default heads module was replaced with the dropout + linear head above. Aliases (fc, layer4) were added so the same Phase A/B training code works across ResNet, EfficientNet, and ViT (layer4 maps to the last encoder block in ViT).

3. Fine-tuning strategy (frozen layers, unfreezing schedule).

I divided training into two phases. The first being phase A where only the classification head is trained while the backbone remains frozen. The second being phase B where an additional portion of the network is unfrozen and fine-tuned, allowing the model to learn task-specific features and improve performance past what was achieved in phase A.

Phase A — Head only (~10–15 epochs):

- Freeze entire backbone
- Train only the classifier head
- Learning rate: $3e-3$ (AdamW)

Phase B — Last block + head (~12–15 epochs):

- Load best Phase A checkpoint
- Unfreeze last backbone block (layer4 / last ViT encoder layer) + head
- Lower learning rates to limit catastrophic forgetting:
- Last block: $3e-5$
- Head: $1e-4$

This staged approach lets the head learn class boundaries first before adjusting deeper features. For ResNet18, Phase B helped (~ 54% Kaggle). For EfficientNet-B3, Phase B often hurt validation accuracy by not improving it and instead overfitting on the training data. ViT-B/16 used the same schedule for the final run and showed improvement in phase B with an increase of ~ 5 - 7%.

3.2 Training

1. Loss function and optimizer.

I used the **Cross Entropy** loss function with label **smoothing = 0.1**. I also used AdamW as the primary optimizer with a weight decay between $1e-3$ to $1e-5$. I tested different weight decay values between $1e-3$ and $1e-5$. With ResNet18, weight decay around $1e-4$ sometimes

helped a bit. But with ViT-B/16 (my current best model) weight decay actually lowered accuracy instead of improving it. My intuition says that with a small dataset, extra regularization from weight decay made it harder for the model to adapt the pretrained features.

I also used MixUp during training with $\alpha = 0.2$. Each batch, two random images are blended together and the loss is computed on both labels proportionally. This creates more training variety and makes it harder for the model to memorize individual images. Since the dataset is small, MixUp was one of the regularization techniques that helped the most.

Finally, I used gradient clipping with a max norm of 1.0. This limits how large the gradients can get during backpropagation, which helps keep training stable when fine-tuning deeper layers in Phase B. Without clipping I sometimes saw loss spikes when unfreezing the last encoder block, especially with a higher learning rate on the head.

2. Learning rate, scheduler, batch size, epochs, weight decay.

Setting	Value
Batch Size	32
Phase A epochs	15
Phase B epochs	15
Phase A LR	3e-3
Phase B block LR	3e-5
Phase B head LR	1e-4
Weight Decay	2e-4 (tested, used during experiments)
Dropout	0.3
Label Smoothing	0.1
MixUp α	0.2

During development, checkpoints were saved on best validation accuracy (80/20 split). For the final Kaggle model, training used all data and checkpoints were saved on best training accuracy. No learning-rate scheduler was used in the final ViT run (fixed LR per phase).

3. Hardware/software environment (GPU, PyTorch version).

For hardware and software, I trained on a MacBook Pro with an Apple M4 chip and 16 GB of memory. Since the project code uses CPU when CUDA is not available, training ran on CPU (not a dedicated NVIDIA GPU). That made each epoch slower, especially for ViT-B/16, but it was enough to run experiments and generate Kaggle submissions.

The software environment was Python with PyTorch 2.12.0 and torchvision 0.27.0. I also used NumPy, Pillow, and matplotlib for data loading and plotting training curves. I set up a virtual environment and installed dependencies from requirements.txt. Pretrained ImageNet weights were downloaded automatically from torchvision on the first training run.

4 Experiments

1. Baseline setup.

- Model: ResNet18, ImageNet pretrained
- Batch size: 64
- Epochs: 15
- Loss: Cross-entropy
- Training: Replace fc layer only, default augmentations (flip, rotation, color jitter)
- Result: ~ 43% validation accuracy with clear overfitting (high train, lower val)

The baseline reached around ~ 43% which confirmed transfer learning works but regularization and careful fine-tuning are needed. After making the baseline I could start experimenting with more advanced techniques such as dropout, different learning rates, more augmentations, and different models.

2. Hyperparameter tuning plan and validation method.

Validation method:

I used a fixed 80/20 train/validation split with **SEED = 42** so results were comparable across runs. I used validation accuracy to decide whether a change helped. Validation was sometimes noisy because the val set is only ~216 images, but it was still useful for comparing learning rates, dropout, and fine-tuning strategies without submitting to Kaggle every time. Before my final Kaggle submission, I retrained on the full training set (all 1,079 images) using the best settings from validation.

Hyperparameters Explored:

Learning rate: 1e-3, 3e-3 (Phase A), 1e-4 and 5e-5 (Phase B), 3e-5 (ViT last encoder block).
Batch size: 32, 64. Optimizer: Adam vs AdamW (final model uses AdamW). Dropout: 0.0 - 0.5 (final: 0.3). Weight decay: 1e-3 to 1e-5 (hurt ViT performance. Disabled in final run). Also fixed: label smoothing (0.1), MixUp ($\alpha = 0.2$).

Steps:

- Fix label mapping (NumericImageFolder)
 - critical first step
- Tune head-only learning rate and weight decay
- Add regularization: dropout, label smoothing, MixUp, stronger augmentation
- Try Phase B (unfreeze last block) with lower LR on backbone vs head
- Compare backbones: ResNet18 $\rightarrow$ EfficientNet-B3 $\rightarrow$ ViT-B/16
- Use 80/20 val split (seed 42) to pick settings. Retrain on full data for final submit
- Evaluate on Kaggle with 10-crop TTA

Validation accuracy was the main metric during development. Kaggle public leaderboard was the final check.

3. Ablations (augmentations, model size, freezing strategy, LR, etc.).

Data augmentation:

I tested stronger augmentation over time: random resized crop, horizontal flip, rotation, color jitter, random grayscale, and random erasing. Stronger augmentation generally helped reduce overfitting and improved validation/Kaggle performance compared to weak or no augmentation.

MixUp:

I added MixUp during training ($\alpha = 0.2$), where two images and labels are blended in each batch. This helped regularization on a small dataset and made the model less likely to memorize exact training images.

Freezing strategy (Phase A vs Phase B):

- Phase A (feature extraction): freeze backbone, train head only
- Phase B (partial fine-tuning): unfreeze last block + head with lower learning rates

For ResNet18, Phase B improved validation (roughly mid-40s to ~56%). For EfficientNet-B3, Phase B often made things worse by increasing overfitting. This showed that unfreezing more layers is not always better on ~1,000 images.

Model size / architecture:

I compared ResNet18, ResNet50, EfficientNet-B3, and ViT-B/16. ResNet18 was a good fast baseline. ResNet50 gave a small Kaggle gain but was much slower. EfficientNet was strong head-only but sensitive in Phase B. ViT-B/16 became my best final model.

Learning rate:

Higher LR on the head ($1e-3$ to $3e-3$) worked in Phase A. For Phase B, lower backbone LR (around $3e-5$ to $5e-5$) with a slightly higher head LR ($1e-4$) gave more stable fine-tuning than using one LR for all unfrozen layers.

Regularization (dropout + label smoothing):

Dropout in the classifier head reduced overfitting; 0.3 worked best in my tests. Label smoothing (0.1) also helped prevent overconfident predictions on small classes.

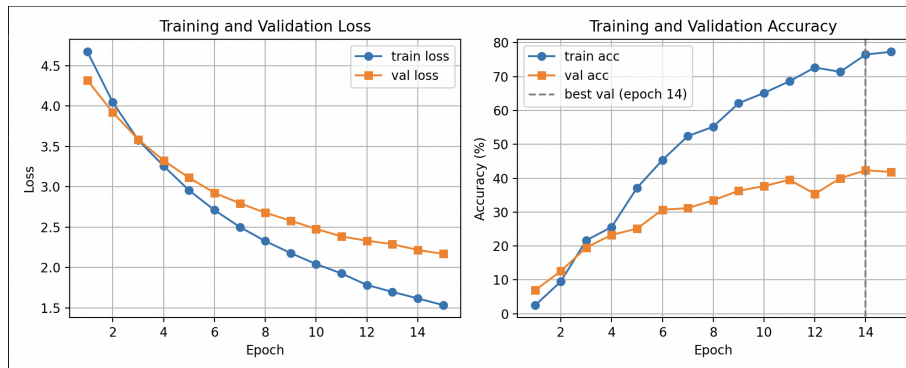
5 Results

1. Training/validation accuracy (and loss) and any key curves/plots (optional).

ResNet18 - head only (early run):

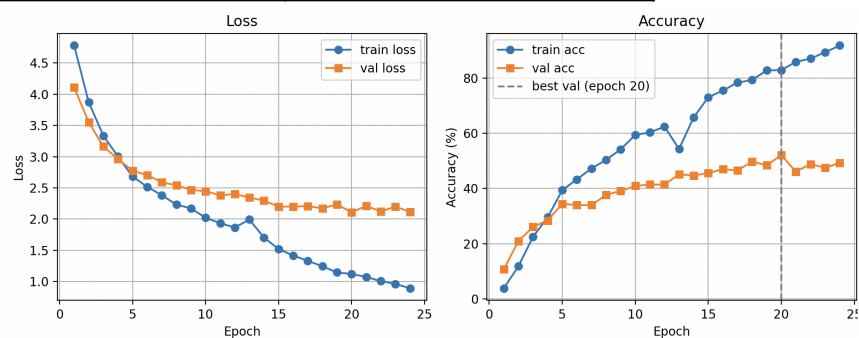
Train loss: 1.29	Train acc: 88.3%
Val loss: 2.18	Best Val acc: 43.3%

Clear overfitting: strong train performance, weak validation.



ResNet18 - head + layer4 (Phase B):

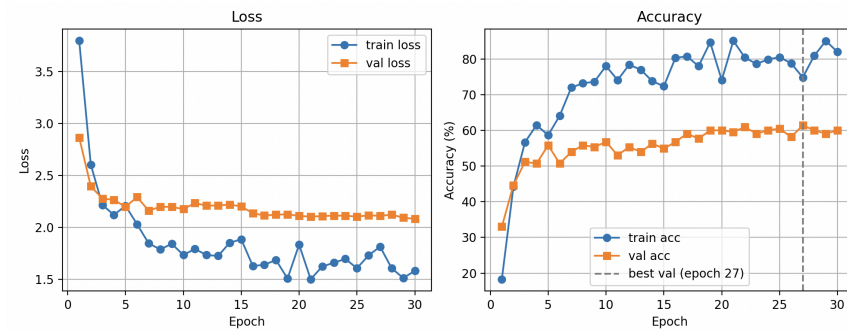
Train loss: 0.8926	Train acc: 91%
Val loss: 2.1	Best Val acc: 52%



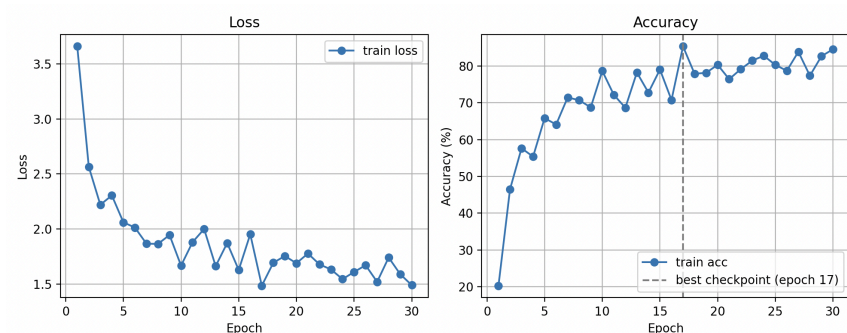
Validation improved, but train hit 91% and the gap widened. Classic overfitting on a small dataset.

ViT-B/16 (80/20 split, Phase A + Phase B):

Train loss: 1.5	Train acc: 85%
Val loss: 2.1	Best Val acc: 61.4%



Graph when all training data is used



General pattern across experiments:

- Train accuracy often reached 90 - 100%
- Validation peaked around ~ 50 - 60% depending on model and phase
- Val accuracy sometimes dropped after the best epoch while train kept improving
- Training curves were saved to ./checkpoints/training_curves.png (loss and accuracy vs epoch, with best checkpoint marked).

2. Kaggle public leaderboard score (and submission settings).

UCSC CSE 144 Spring 2026 Final Project

Overview Data Code Models Discussion **Leaderboard** Rules Team Submissions

Submit Prediction ...

Rank	Participant	Score	Attempts	Time
46	ALYSSA CHIKISIMBI	0.70000	1	7d
47	Satvik Khanna	0.70000	1	7d
48	AlexanderSantiago	0.70000	6	6h
🤖 Your Best Entry! Your most recent submission scored 0.70000, which is the same as your previous score. Keep trying!				
49	admargul	0.66363	4	6h

For the final and best submission ViT-B/16 was used with these parameters:

Setting	Value
Dropout	0.3
Phase A epochs	15
Phase B epochs	15
Total epochs	30
Phase A LR	3e-3
Phase B LR (head)	1e-4
Phase B LR (last encoder block)	3e-5

6 Discussion

1. What worked best and why.

The most reliable gains came from transfer learning with ImageNet-pretrained backbones, a consistent preprocessing pipeline (resize, crop, ImageNet normalization), and a clear train/validation workflow. I used an 80/20 split to compare hyperparameters locally, then retrained on the full training set before my final Kaggle submission. Among the models I tried (ResNet18, EfficientNet-B3, ResNet50, ViT-B/16), ViT-B/16 gave the strongest performance, with about 61% validation accuracy and my best Kaggle score of 70%. A two-phase fine-tuning strategy also helped: train the head first, then unfreeze only the last encoder block with a lower learning rate instead of fine-tuning the whole network at once. Strong regularization mattered on this small dataset (~ 10 images per class): dropout (0.3), label smoothing, MixUp, and aggressive augmentation (color jitter, random erasing, random crops) reduced overfitting and improved generalization compared to training with weak or no augmentation.

2. Failure cases, overfitting/underfitting observations.

The main issue was overfitting. Train accuracy neared 95% while validation plateaued or dropped around ~ 57%. Unfreezing more layers did not always help:

- ResNet18 Phase B: modest val gain, large train/val gap
- EfficientNet-B3 Phase B: often made things worse

More unfrozen parameters lead to a larger capacity to memorize ~ 864 training images. Validation accuracy also stalled or decreased after the best epoch if training continued too long - saving the best checkpoint (not the last epoch) was important.

3. Limitations and concrete next improvements.

Limitations:

- Very small dataset (~ 10 images/class) limits how much any model can generalize
- 80/20 val split is noisy with only ~ 215 validation images
- ViT is slower on CPU. 10-crop TTA adds further inference time

Concrete next steps:

- K-fold cross-validation (e.g. 5-fold) for more reliable tuning and model ensembling
- Stronger weight decay / early stopping on validation during Phase B
- Multi-model ensembling across the top architectures from different families (e.g., ViT-B/16 + ResNet-50 + etc.) can lead to better results
- Implementing CutMix for more image diversity, leading to more training images

7 Reproducibility

1. Random seeds and determinism settings.

- Global seed: SEED = 42
- Seeds set for: Python random, NumPy, PyTorch (CPU and CUDA)
- `torch.backends.cudnn.deterministic = True, benchmark = False`
- Train/val split uses `torch.Generator().manual_seed(42)`
- Train DataLoader shuffle uses a fixed generator
- `num_workers = 0` for consistent ordering across platforms

I based the project's consistency on Assignment 2 and the settings used. The code expects the dataset in a `data/` directory with `train/` (class folders 0–99) and `test/` (flat .jpg files). Trained model checkpoints are saved to `checkpoints/`, and `src/inference.py` generates `submission.csv` for Kaggle.

2. Package versions / environment setup steps.

Versions in **requirements.txt**

- torch==2.12.0
- torchvision==0.27.0
- numpy==2.4.6
- pillow==12.2.0
- matplotlib==3.10.9

Environment setup steps

```
$ python -m venv venv  
$ source venv/bin/activate  
$ pip install -r requirements.txt
```

3. Exact commands to train and to generate submission.csv.

Training

```
$ source venv/bin/activate  
$ PYTHONPATH=src python src/train.py
```

Inference

```
$ source venv/bin/activate  
$ PYTHONPATH=src python src/inference.py
```

Trained Model Weights

https://drive.google.com/file/d/1ZzEGcsp8URlukgSYsX6aQ-08ZPTlGhL/view?usp=drive_link

Quick Note

Checkpoints are not required in the repo, they are produced by running train.py. Pretrained ImageNet weights download automatically on the first run.

8 Team Contributions

N/A

9 References

1. TorchVision documentation / pretrained model sources.

TorchVision Documentation <https://docs.pytorch.org/vision/stable/index.html>

ResNet18 Documentation: [link](#)

PyTorch Documentation: [link](#)

Efficientnet_b3: [link](#)

Vit_b_16: [link](#)

2. Any papers or tutorials used.

Fine-Tuning a Pre-Trained ResNet-18 Model: [link](#)